

WEB4 PHP - Game of Thrones Quotes

Nathalie Sonnleitner

Tim Peko

Inhaltsverzeichnis

1. Einleitung	2
2. Projektmitglieder	2
3. Anforderungsübersicht	2
3.1. Rollenmodell	2
4. Datenmodell	3
4.1. ER-Diagramm	3
4.2. Tabellen	3
4.2.1. users	3
4.2.2. quotes	4
4.2.3. comments	4
4.2.4. quote_likes	4
4.2.5. comment_votes	4
4.3. Testdaten	4
5. Session und Login	5
6. Architektur	5
6.1. MVC-Schichten	5
6.2. Verzeichnisstruktur	6
6.3. Request-Flow (Beispiel: Kommentar löschen)	6
6.4. REST-Routing (Auszug)	6
7. Sicherheitskonzept	6
8. Testfälle	7
8.1. Test-Runner	7
8.2. Automatisierte REST-Tests (httpYac)	7
8.2.1. Übersicht	7
8.2.2. Erwartete und beobachtete Werte	8
8.3. Manuelle UI-Tests	9
8.4. Übersicht	9
8.5. Detailergebnisse	9
9. Installation	11
9.1. Voraussetzungen	11
9.2. Schritte	11

1. Einleitung

Diese Dokumentation beschreibt die Webanwendung **GoT Quotes**. Es ist eine PHP-Anwendung im MVC-Pattern, die berühmte Game-of-Thrones-Zitate als **Foren-Posts** präsentiert. Eingeloggte Nutzer diskutieren in **threadartigen Kommentarbäumen** (Antworten auf Antworten). Administratoren verwalten Benutzer, Zitate inkl. Bild-Uploads; Nutzer können optionale Profilbilder setzen.

Start-URL (XAMPP): `http://localhost/`. Document Root muss auf das Verzeichnis `public/` zeigen.

2. Projektmitglieder

Name	Matrikelnummer
Nathalie Sonnleitner	-
Tim Peko	s2420458029

3. Anforderungsübersicht

Die Anwendung erfüllt die Pflichtenanforderungen der Projektangabe:

- MVC-Architektur mit PHP 8.x und PDO
- Rollenbasierte Interaktion (Gast, User, Admin)
- Registrierung und Login (Session-basiert, kein Auto-Login nach Registrierung)
- Vollständiges CRUD für Kommentare (Haupt-Ressource) und Zitate (Admin)
- Benutzerverwaltung für Admins (Löschen, Rolle ändern)
- Serverseitige Validierung, SQL-Injection- und XSS-Schutz
- Valides HTML5, Tailwind CSS 4 (CDN), JavaScript nur für REST-DELETE/PATCH (Fetch)
- REST-konforme HTTP-Methoden (GET/POST/PUT/PATCH/DELETE)
- Thread-Kommentare, Bild-Uploads (Zitate/Avatare), Likes und Votes
- Feed-Sortierung (neu/top/trending), öffentliche Nutzerprofile

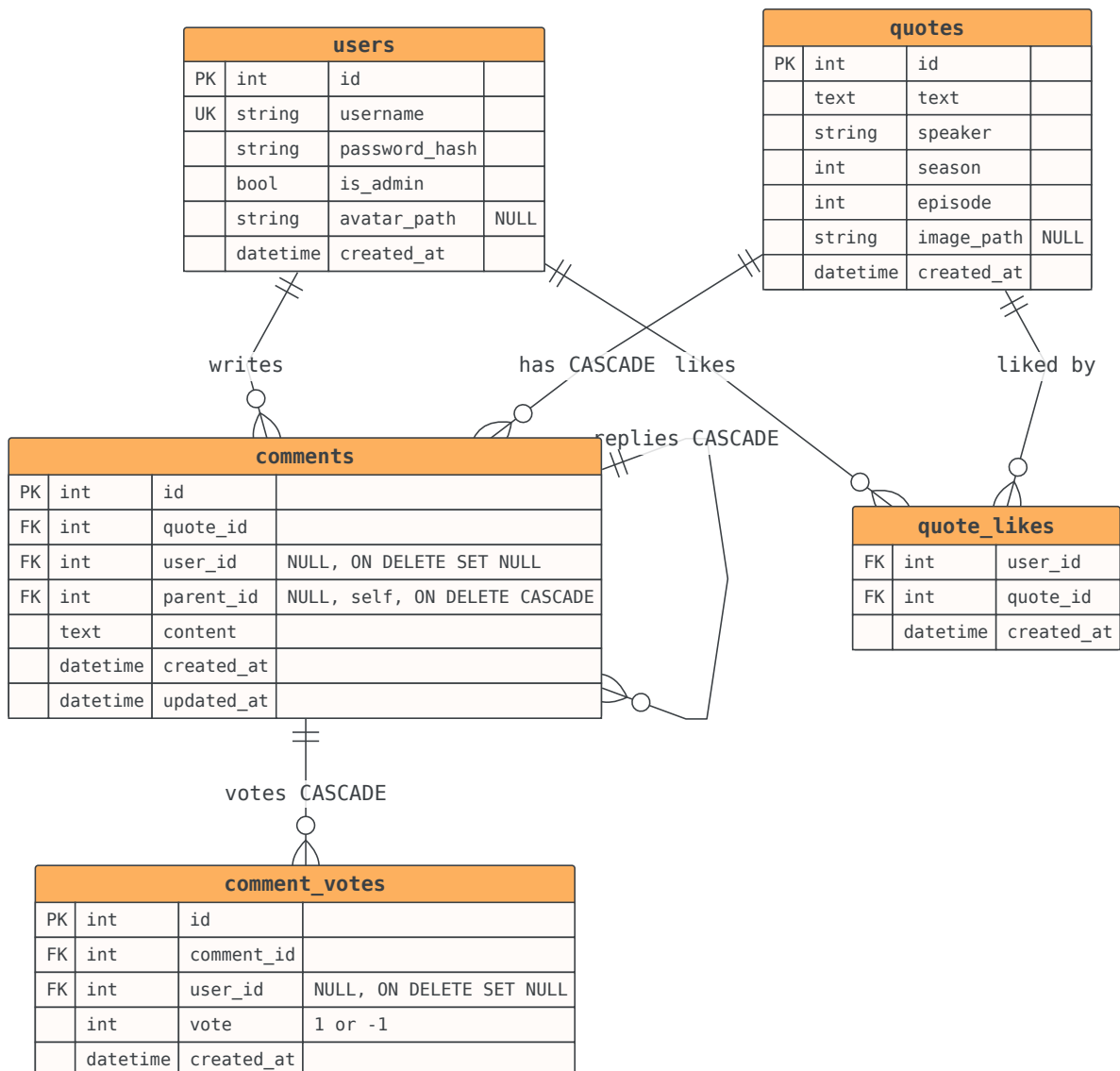
3.1. Rollenmodell

Aktion	Gast	User	Admin
Zitate & Kommentare lesen	✓	✓	✓
Kommentar schreiben / eigenes bearbeiten & löschen	-	✓	✓
Fremden Kommentar löschen	-	-	✓
Fremden Kommentar bearbeiten	-	-	X
Zitate verwalten (CRUD)	-	-	✓
Benutzerverwaltung	-	-	✓

Bei gelöschten Benutzern bleiben Kommentare erhalten; `user_id` wird auf `NULL` gesetzt und in der UI als graues `<deleted>` angezeigt.

4. Datenmodell

4.1. ER-Diagramm



4.2. Tabellen

4.2.1. users

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
is_admin	TINYINT(1)	NOT NULL, DEFAULT 0
avatar_path	VARCHAR(255)	NULL, Profilbild unter /uploads/avatars/
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

4.2.2. quotes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
text	TEXT	NOT NULL
speaker	VARCHAR(100)	NOT NULL
season / episode	TINYINT UNSIGNED	NULL, optional
image_path	VARCHAR(255)	NULL, Beitragsbild unter /uploads/quotes/
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

4.2.3. comments

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
parent_id	INT UNSIGNED	FK → comments.id, NULL, ON DELETE CASCADE
content	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	NULL ON UPDATE

4.2.4. quote_likes

Spalte	Typ	Constraints
user_id	INT UNSIGNED	FK → users.id, ON DELETE CASCADE
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Primärschlüssel: (user_id, quote_id) – ein Like pro Nutzer und Zitat.

4.2.5. comment_votes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
comment_id	INT UNSIGNED	FK → comments.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
vote	TINYINT	NOT NULL, Werte 1 oder -1
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

4.3. Testdaten

SQL-Dump: WEB4_PHP_TEAM7.sql

• Admin: admin / admin

- Testuser: `tyrion_fan`, `arya_fan` (Passwort: `password123`)
- 12 Zitate, 17 Kommentare (inkl. verschachtelter Antworten), Likes und Votes

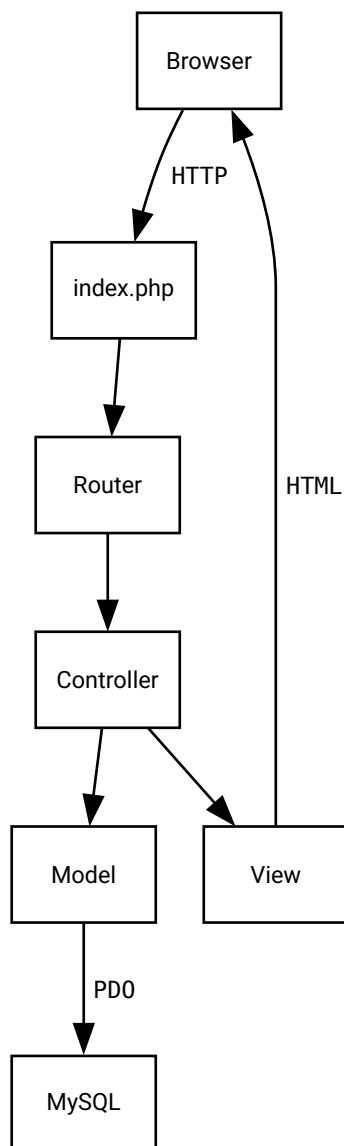
5. Session und Login

Login-Status liegt in der PHP-Session (`$_SESSION`: `user_id`, `username`, `is_admin`, `avatar_path`). `AuthService` kapselt Login, Logout und Berechtigungsprüfungen. Flash-Messages werden nach Redirect einmalig angezeigt.

Verwendete PHP-Konzepte: PDO mit Prepared Statements, Sessions, `password_hash` / `password_verify`, `htmlspecialchars`, MVC mit `include` / `require`.

6. Architektur

6.1. MVC-Schichten



6.2. Verzeichnisstruktur

1	public/	
2	├─ index.php	# Front Controller, REST-Routing
3	├─ .htaccess	# URL Rewriting
4	├─ assets/js/app.js	# Fetch DELETE/PATCH
5	├─ uploads/	# Avatare & Zitatbilder
6	├─ src/	
7	│ └─ bootstrap.php	
8	│ └─ config.php	
9	│ └─ Router.php	# GET, POST, PUT, PATCH, DELETE
10	│ └─ Controller/	# Auth, Profile, Quote, Comment, Admin/*
11	│ └─ Model/	# User, Quote, Comment, QuoteLike, CommentVote
12	│ └─ Service/	# AuthService, ValidationService, UploadService
13	└─ views/	# PHP-Templates (Tailwind CDN)

6.3. Request-Flow (Beispiel: Kommentar löschen)

1. `DELETE /comments/{id}` → Router (Fetch aus `app.js`)
2. `CommentController::destroy`: CSRF, Login, Berechtigung
3. `Comment::delete` mit CASCADE auf Antworten
4. Redirect zur Zitat-Detailseite

6.4. REST-Routing (Auszug)

Methode	Pfad	Aktion
GET	/	Zitate-Feed (<code>?sort=new top trending</code>)
POST/DELETE	/quotes/{id}/likes	Zitat liken / Like entfernen
POST/DELETE	/comments/{id}/votes	Kommentar bewerten
GET	/users/{id}	Öffentliches Profil
POST	/quotes/{id}/comments	Top-Level-Kommentar
POST	/comments/{id}/replies	Thread-Antwort
DELETE	/comments/{id}	Kommentar löschen (Fetch)
DELETE	/admin/quotes/{id}	Zitat löschen (Fetch)
PATCH	/admin/users/{id}/admin	Admin-Rolle toggeln (Fetch)

7. Sicherheitskonzept

Bedrohung	Maßnahme
SQL Injection	PDO Prepared Statements für alle Queries mit User-Input
XSS	<code>htmlspecialchars()</code> bei jeder Ausgabe usergenerierter Inhalte
Session Fixation	<code>session_regenerate_id()</code> nach Login
CSRF	Token in allen POST-Formularen, Validierung in Controllern

Bedrohung	Maßnahme
IDOR	Berechtigungsprüfung: Kommentar-Bearbeitung nur für Autor
Passwörter	<code>password_hash()</code> / <code>password_verify()</code> , PASSWORD_DEFAULT
REST / JS	DELETE und PATCH per Fetch + CSRF-Header; Validierung nur serverseitig
Uploads	MIME-Check via finfo, Größenlimit, <code>/uploads/.htaccess</code> ohne Script-Ausführung

8. Testfälle

Die REST-Schnittstelle wird automatisiert mit **httpYac** getestet. httpYac ist ein CLI-Runner für `.rest`-Dateien (kompatibel mit IntelliJ HTTP Client / VS Code REST Client). Jede Anfrage enthält Assertions (`?? status = 200`, `?? body includes ...`), die Statuscode und Response-Body prüfen.

8.1. Test-Runner

Tool	Verwendung
httpYac CLI	<code>npm install -g httpyac</code> oder via <code>mise install</code>
Tests ausführen	<code>mise run test:rest</code>
JUnit-Report	<code>doc/test-results/httpyac-junit.xml</code>
Testdateien	<code>tests/rest/*.rest</code>

Voraussetzungen: PHP-Server und MySQL laufen bereits (z. B. `mise run run:app` und `mise run run:db`), SQL-Dump `WEB4_PHP_TEAM7.sql` importiert. Standard-URL:
`http://127.0.0.1:8080`.

8.2. Automatisierte REST-Tests (httpYac)

Die folgenden Tabellen wurden aus dem JUnit-Report generiert. Spalte **Erwartet** stammt aus der Assertion in der `.rest`-Datei; **Beobachtet** ist der tatsächliche Wert (bei Fehlern aus der httpYac-Fehlermeldung).

8.2.1. Übersicht

Testsuite	Tests	Fehler
Total	47	0
<code>\tests\rest\01-public.rest</code>	11	0
<code>\tests\rest\02-auth.rest</code>	11	0
<code>\tests\rest\03-comments.rest</code>	11	0
<code>\tests\rest\04-admin.rest</code>	7	0
<code>\tests\rest\05-forum.rest</code>	7	0

8.2.2. Erwartete und beobachtete Werte

Request	Prüfung	Erwartet	Beobachtet	Status
\tests\rest\01-public.rest				
pubFeed	status == 200	200	200	OK
pubFeed	body contains Zitate			OK
pubQuoteDetail	status == 200	200	200	OK
pubQuoteDetail	body contains Cersei Lannister			OK
pubQuote404	status == 404	404	404	OK
pubProfile	status == 200	200	200	OK
pubProfile	body contains tyrion_fan			OK
pubSortTop	status == 200	200	200	OK
pubSortTop	body contains sort=top			OK
pubCommentSort	status == 200	200	200	OK
pubAdminForbidden	status == 403	403	403	OK
\tests\rest\02-auth.rest				
authLoginForm	status == 200	200	200	OK
authLoginForm	body contains Einloggen			OK
authLoginOk	status == 302	302	302	OK
authLoginOk	location == /	/	/	OK
authSessionCheck	status == 200	200	200	OK
authSessionCheck	body contains tyrion_fan			OK
authLogout	status == 302	302	302	OK
authProfileRedirect	status == 302	302	302	OK
authProfileRedirect	location == /login	/login	/login	OK
authLoginFail	status == 200	200	200	OK
authLoginFail	body contains ungültig			OK
\tests\rest\03-comments.rest				
cmtCreate	status == 302	302	302	OK
cmtShow	status == 200	200	200	OK
cmtShow	body contains Automatisierter REST-Test			OK
cmtForbiddenEdit	status == 403	403	403	OK
cmtEditForm	status == 200	200	200	OK
cmtEditForm	body contains Kommentar bearbeiten			OK
cmtUpdate	status == 302	302	302	OK
cmtUpdated	status == 200	200	200	OK
cmtUpdated	body contains REST updated			OK
cmtEmptyRejected	status == 200	200	200	OK

Request	Prüfung	Erwartet	Beobachtet	Status
\tests\rest\03-comments.rest				
cmtEmptyRejected	body contains darf nicht leer			OK
\tests\rest\04-admin.rest				
admUserList	status == 200	200	200	OK
admUserList	body contains Benutzer			OK
admQuoteList	status == 200	200	200	OK
admQuoteList	body contains Zitate			OK
admQuoteNewForm	status == 200	200	200	OK
admQuoteNewForm	body contains Anlegen			OK
admUserForbidden	status == 403	403	403	OK
\tests\rest\05-forum.rest				
forumLike	status == 302	302	302	OK
forumUnlike	status == 302	302	302	OK
forumUpvote	status == 302	302	302	OK
forumRemoveVote	status == 302	302	302	OK
forumDeleteNoCsrf	status == 302	302	302	OK
forumDeleteNoCsrf	location == /	/	/	OK
forumBadLike	status == 404	404	404	OK

8.3. Manuelle UI-Tests

Zusätzlich wurden folgende Szenarien manuell in Google Chrome geprüft (Uploads, responsives Layout, Thread-Darstellung):

8.4. Übersicht

Testsuite	Tests	Fehler
Total	32	0
Authentifizierung	6	0
Zitate & Kommentare	9	0
Administration	8	0
Forum & Interaktion	6	0
Sicherheit	3	0

8.5. Detailliergebnisse

Testname	Status	Zeit
Authentifizierung		
T-AUTH-01: Registrierung gültig → Erfolg, Redirect Login	✓ Erfolgreich	manuell
T-AUTH-02: Duplicate Username → Fehler	✓ Erfolgreich	manuell
T-AUTH-03: Passwort < 8 Zeichen → Fehler	✓ Erfolgreich	manuell

Testname	Status	Zeit
Authentifizierung		
T-AUTH-04: Login korrekt → Session aktiv	✓ Erfolgreich	manuell
T-AUTH-05: Login falsch → generische Fehlermeldung	✓ Erfolgreich	manuell
T-AUTH-06: Logout → Session beendet	✓ Erfolgreich	manuell
Zitate & Kommentare		
T-QUOTE-01: Gast sieht Liste und Detail	✓ Erfolgreich	manuell
T-CMT-01: Gast sieht Kommentare, kein Formular	✓ Erfolgreich	manuell
T-CMT-02: User erstellt Kommentar mit Username	✓ Erfolgreich	manuell
T-CMT-03: User bearbeitet eigenen Kommentar	✓ Erfolgreich	manuell
T-CMT-04: User löscht eigenen Kommentar	✓ Erfolgreich	manuell
T-CMT-05: Fremdes Edit → 403	✓ Erfolgreich	manuell
T-CMT-06: Fremdes Delete → 403	✓ Erfolgreich	manuell
T-CMT-07: Leerer Kommentar → Validierungsfehler	✓ Erfolgreich	manuell
T-CMT-08: XSS-Payload → escaped in Ausgabe	✓ Erfolgreich	manuell
Administration		
T-ADM-01: Admin löscht fremden Kommentar	✓ Erfolgreich	manuell
T-ADM-02: Admin kann fremden Kommentar nicht bearbeiten	✓ Erfolgreich	manuell
T-ADM-03: Admin togglet User-Rolle	✓ Erfolgreich	manuell
T-ADM-04: Admin löscht User → Kommentare zeigen <deleted>	✓ Erfolgreich	manuell
T-ADM-05: Nicht-Admin auf /admin → 403	✓ Erfolgreich	manuell
T-ADM-06: Letzter Admin nicht degradierbar	✓ Erfolgreich	manuell
T-ADM-07: Admin CRUD Zitate	✓ Erfolgreich	manuell
T-ADM-08: Admin kann eigene Rolle nicht entziehen	✓ Erfolgreich	manuell
Forum & Interaktion		
T-FORUM-01: Feed-Sortierung top/trending	✓ Erfolgreich	manuell
T-FORUM-02: Zitat liken und entliken	✓ Erfolgreich	manuell
T-FORUM-03: Kommentar up-/downvoten	✓ Erfolgreich	manuell
T-FORUM-04: Kommentar-Sortierung auf Detailseite	✓ Erfolgreich	manuell
T-FORUM-05: Öffentliches Profil mit Kommentaren und Likes	✓ Erfolgreich	manuell
T-FORUM-06: Thread-Antworten und CASCADE-Löschung	✓ Erfolgreich	manuell
Sicherheit		
T-SEC-01: SQL-Injection Login → kein Bypass	✓ Erfolgreich	manuell
T-SEC-02: Direktaufruf fremdes Edit → 403	✓ Erfolgreich	manuell
T-SEC-03: Ungültige Quote-ID → 404	✓ Erfolgreich	manuell

9. Installation

9.1. Voraussetzungen

- XAMPP (Apache + PHP 8.x + MySQL/MariaDB)

9.2. Schritte

1. ZIP entpacken
2. In phpMyAdmin `WEB4_PHP_TEAM7.sql` importieren
3. Apache DocumentRoot auf `public/` setzen
4. Anwendung unter `http://localhost/` aufrufen
5. Mit `admin` / `admin` einloggen

Datenbank: `team_7`, User `fh_webphp`, Passwort `fh_webphp`.